Bài 2.18 – Tinh chỉnh tham số: Grid Search, Random Search

> **Giai đoạn 2 – Machine Learning cổ điển** · Thời lượng: 1 buổi (2–3 giờ) · Hình thức mở đầu: *Thử thách có tính giờ*

🎯 Sau bài này bạn sẽ

- Phân biệt **tham số** (máy tự học) và **siêu tham số** (bạn phải chọn).
- Tinh chỉnh tự động bằng **GridSearchCV** và **RandomizedSearchCV**, kết hợp cross-validation.
- Hiểu vì sao **Random Search thường hiệu quả hơn Grid Search**.
- Chọn **phạm vi** tìm kiếm hợp lý (thang log) và biết các bẫy khi tinh chỉnh.

---

1. Thử thách có tính giờ ⏱️ (3 phút, chỉ cần giấy bút)

Bạn muốn tinh chỉnh LightGBM với **5 siêu tham số**, mỗi tham số thử **5 giá trị**. Đánh giá bằng **5-fold cross-validation**. Mỗi lần huấn luyện mất **2 giây**.

Câu hỏi:

1. Có bao nhiêu tổ hợp?
2. Tổng cộng phải huấn luyện bao nhiêu lần?
3. Mất bao lâu?

Bấm giờ!

Đáp án

> Mẹo Notion: chọn tiêu đề "Đáp án" → `Turn into → Toggle heading 3`.

1. $5 \times 5 \times 5 \times 5 \times 5 = 5^5 =$ **3.125 tổ hợp**.
2. 3.125 × 5 fold = **15.625 lần huấn luyện**.
3. 15.625 × 2 giây = 31.250 giây ≈ **8,7 giờ**.

Thêm một tham số nữa (5 giá trị): **43 giờ**. Thử từng tổ hợp (Grid Search) "bùng nổ" rất nhanh. Bài này dạy cách thông minh hơn: thử **30 tổ hợp ngẫu nhiên** trong khoảng 1 phút, kết quả thường không kém.

---

2. Tham số và siêu tham số

	Tham số (parameter)	**Siêu tham số (hyperparameter)**
Ai quyết định	**Máy tự học** từ dữ liệu khi `fit`	**Bạn chọn** trước khi `fit`
Ví dụ	w, b của Linear Regression; ngưỡng chia của từng nút cây	alpha của Ridge; K của KNN; C, gamma của SVM; max_depth, learning_rate

	Tham số (parameter)	Siêu tham số (hyperparameter)
Ví von	Kiến thức học sinh học được	Phương pháp học: học mấy giờ/ngày, học nhóm hay một mình

Bạn đã chọn siêu tham số bằng tay nhiều lần: K của KNN (bài 2.7), alpha (bài 2.6), C và gamma (bài 2.12)... Và nhiều lần bạn được nhắc: **"chọn trên tập test là gian lận"**. Giờ là lúc làm đúng.

Quy trình đúng: thử nhiều bộ siêu tham số → đánh giá mỗi bộ bằng **cross-validation trên train** → chọn bộ tốt nhất → huấn luyện lại trên toàn bộ train → kiểm tra trên test **một lần**.

---

3. Grid Search (Tìm kiếm lưới)

3.1. Khái niệm

- **Định nghĩa:** Liệt kê danh sách giá trị cho mỗi siêu tham số, thử **mọi tổ hợp**.
- **Ví dụ đời thực:** Thử mọi cách phối 4 loại áo với 4 loại quần để chọn bộ đẹp nhất: 16 bộ.

3.2. Tinh chỉnh SVM cho dữ liệu ung thư vú

```
import time
import numpy as np
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV,
StratifiedKFold, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import roc_auc_score

bc = load_breast_cancer(as_frame=True)
Xtr, Xte, ytr, yte = train_test_split(bc.data, bc.target, test_size=0.25,
random_state=42, stratify=bc.target)

pipe = Pipeline([("chuan_hoa", StandardScaler()), ("svm", SVC())])
print(sorted(pipe.get_params().keys())[:6])        # xem tên các siêu tham số có
thể chỉnh
```

Kết quả mẫu:

```
['chuan_hoa', 'chuan_hoa__copy', 'chuan_hoa__with_mean', 'chuan_hoa__with_std',
'memory', 'steps']
```

Quy tắc đặt tên trong pipeline: `<tên bước>__<tên siêu tham số>` (**hai** dấu gạch dưới). Ví dụ `svm__C` là tham số C của bước tên "svm".

```
luoi = {"svm__C": [0.1, 1, 10, 100],                # thang log: mỗi giá trị gấp
10 lần
        "svm__gamma": [0.001, 0.01, 0.1, 1]}
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

t0 = time.time()
gs = GridSearchCV(pipe, luoi, cv=cv, scoring="roc_auc", n_jobs=-1,   # n_jobs=-1:
chạy song song
                  return_train_score=True).fit(Xtr, ytr)
print(f"{time.time() - t0:.1f} giây | Tốt nhất: {gs.best_params_} | AUC CV =
{gs.best_score_:.4f}")
```

Kết quả mẫu:

```
4.1 giây | Tốt nhất: {'svm__C': 10, 'svm__gamma': 0.01} | AUC CV = 0.9971
```

4 × 4 = 16 tổ hợp × 5 fold = **80 lần huấn luyện**.

3.3. Xem toàn bộ kết quả

```
kq = pd.DataFrame(gs.cv_results_)          # mọi tổ hợp đã thử, cùng điểm từng
fold
print(kq.pivot_table(index="param_svm__C", columns="param_svm__gamma",
values="mean_test_score").round(4))
```

Kết quả mẫu:

```
param_svm__gamma   0.001   0.010   0.100   1.000
param_svm__C
0.1               0.9859  0.9893  0.9810  0.9506
1.0               0.9914  0.9957  0.9885  0.9507
10.0              0.9956  0.9971  0.9860  0.9508
100.0             0.9962  0.9944  0.9857  0.9508
```

Đọc "bản đồ nhiệt" này:

- gamma = 1 (cột cuối): tệ ở mọi C (~0,95) → đường biên quá lởm chởm, **overfit** (bài 2.12).
- Vùng tốt nằm ở gamma nhỏ (0,001–0,01) và C vừa đến lớn.
- Điểm tốt nhất (C = 10, gamma = 0,01) nằm **bên trong** lưới, không phải ở rìa. Nếu điểm tốt nhất nằm ở **rìa** lưới (ví dụ C = 100 là giá trị lớn nhất bạn thử), hãy **mở rộng lưới** về phía đó: có thể còn giá trị tốt hơn bên ngoài.

3.4. Mô hình cuối cùng và đánh giá trên test

`GridSearchCV` mặc định (`refit=True`) **tự huấn luyện lại** bộ tốt nhất trên **toàn bộ train**. Dùng ngay `gs` như một mô hình:

```
print("AUC test (tinh chỉnh):", round(roc_auc_score(yte,
gs.decision_function(Xte)), 4))

mac_dinh = Pipeline([("chuan_hoa", StandardScaler()), ("svm", SVC())])
print("AUC CV  (mặc định):  ", round(cross_val_score(mac_dinh, Xtr, ytr, cv=cv,
scoring="roc_auc").mean(), 4))
mac_dinh.fit(Xtr, ytr)
print("AUC test (mặc định): ", round(roc_auc_score(yte,
mac_dinh.decision_function(Xte)), 4))
```

Kết quả mẫu:

```
AUC test (tinh chỉnh): 0.9987
AUC CV  (mặc định):   0.9951
AUC test (mặc định):  0.9969
```

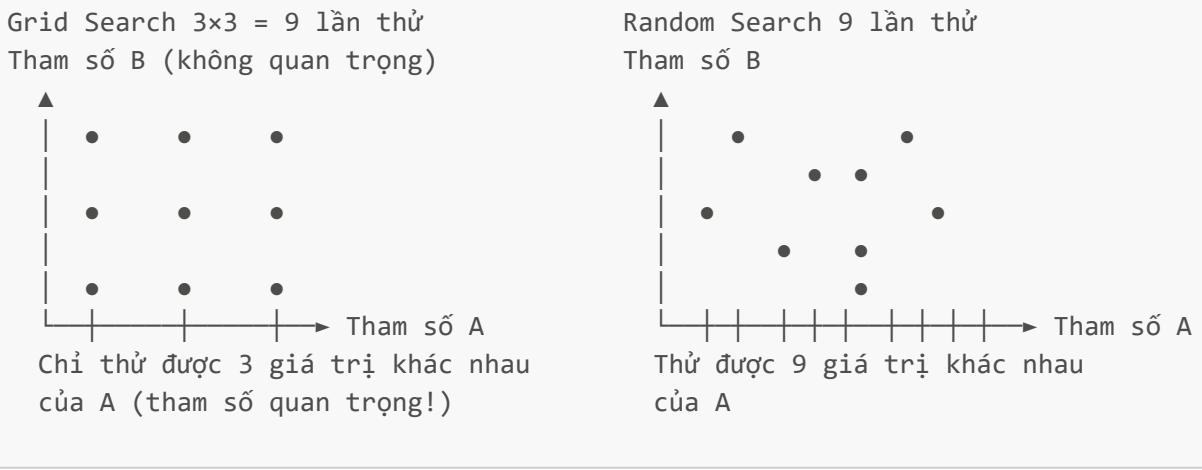
Tinh chỉnh cải thiện nhẹ (CV: 0,9951 → 0,9971; test: 0,9969 → 0,9987). Mô hình mặc định đã rất tốt nên không còn nhiều chỗ để cải thiện. **Đừng kỳ vọng tinh chỉnh tạo phép màu**: đặc trưng tốt (bài 2.16) và chọn đúng loại mô hình thường quan trọng hơn nhiều.

---

4. Random Search (Tìm kiếm ngẫu nhiên)

4.1. Vì sao ngẫu nhiên lại tốt hơn?

Thường chỉ **vài** siêu tham số thật sự quan trọng, nhưng bạn không biết trước là cái nào.



Cùng 9 lần thử, Random Search khám phá được **9 giá trị khác nhau** của tham số quan trọng, Grid Search chỉ được 3. (Kết quả nổi tiếng của Bergstra và Bengio, 2012.)

4.2. Phân phối thang log

Với các tham số như learning_rate, C, alpha, khoảng 0,001–0,01 quan trọng **ngang** khoảng 0,1–1. Lấy mẫu **đều theo thang log** để mỗi "bậc mười" được thử như nhau:

```
from scipy.stats import loguniform, randint, uniform

print(np.sort(loguniform(1e-3, 1e2).rvs(size=8, random_state=0)).round(4))   # 8
giá trị từ 0,001 đến 100
```

Kết quả mẫu:

```
[ 0.1313  0.1541  0.5302  0.5547  1.0323  1.6962  3.7666 28.765 ]
```

(Với 8 mẫu ngẫu nhiên thì phân bố chưa đều hoàn toàn; với nhiều mẫu hơn, mỗi khoảng 0,001–0,01, 0,01–0,1... sẽ có số mẫu gần bằng nhau.)

Phân phối	Dùng cho	Ví dụ
`loguniform(a, b)`	Tham số "nhân", trải nhiều bậc mười	learning_rate, C, alpha, gamma
`randint(a, b)`	Số nguyên	num_leaves, max_depth, min_child_samples
`uniform(a, khoảng)`	Số thực trong **[a, a + khoảng]**	subsample: `uniform(0.6, 0.4)` → từ 0,6 đến 1,0

4.3. Tinh chỉnh LightGBM cho giá nhà

```
from sklearn.model_selection import RandomizedSearchCV, KFold
from sklearn.metrics import mean_absolute_error
import lightgbm as lgb

df = nap_du_lieu_nha()           # ô chuẩn bị dữ liệu ở bài 2.1
cot = ["thu_nhap", "km_tp_lon", "phong_moi_ho", "ti_le_phong_ngu",
       "nguoi_moi_ho", "tuoi_nha", "kinh_do", "vi_do"]
Xtr, Xte, ytr, yte = train_test_split(df[cot], df["gia_nha"], test_size=0.2,
random_state=42)

pham_vi = {
    "learning_rate": loguniform(0.01, 0.3),
    "num_leaves": randint(8, 128),
    "min_child_samples": randint(5, 100),
    "subsample": uniform(0.6, 0.4),
    "colsample_bytree": uniform(0.6, 0.4),
    "reg_lambda": loguniform(1e-3, 10),
}
cv_r = KFold(n_splits=5, shuffle=True, random_state=42)
t0 = time.time()
rs = RandomizedSearchCV(lgb.LGBMRegressor(n_estimators=600, subsample_freq=1,
```

```
random_state=42, verbose=-1),
                        pham_vi, n_iter=30,                        # chỉ thử 30 tổ
hợp ngẫu nhiên
                        cv=cv_r, scoring="neg_mean_absolute_error",    # sklearn
luôn "càng lớn càng tốt" → MAE âm
                        random_state=42, n_jobs=-1).fit(Xtr, ytr)
print(f"{time.time() - t0:.0f} giây")
print({k: (float(round(v, 4)) if isinstance(v, float) else v) for k, v in
rs.best_params_.items()})
print("MAE CV tốt nhất:", round(-rs.best_score_, 2))
```

Kết quả mẫu (khoảng 1–2 phút):

```
79 giây
{'colsample_bytree': 0.6002, 'learning_rate': 0.0332, 'min_child_samples': 7,
'num_leaves': 110, 'reg_lambda': 0.0046, 'subsample': 0.8136}
MAE CV tốt nhất: 26.49
```

30 tổ hợp × 5 fold = **150 lần huấn luyện**. Nếu dùng lưới 5 giá trị cho mỗi tham số trong 6 tham số này: 5^6 = 15.625 tổ hợp × 5 fold = **78.125 lần**, gấp hơn 500 lần.

4.4. So sánh với mặc định

```
mac_dinh = lgb.LGBMRegressor(n_estimators=600, random_state=42, verbose=-1)
print("MAE CV mặc định:", round(-cross_val_score(mac_dinh, Xtr, ytr, cv=cv_r,
scoring="neg_mean_absolute_error").mean(), 2))
mac_dinh.fit(Xtr, ytr)
print("MAE test mặc định:", round(mean_absolute_error(yte, mac_dinh.predict(Xte)),
2),
      "| MAE test tinh chỉnh:", round(mean_absolute_error(yte, rs.predict(Xte)),
2))
```

Kết quả mẫu:

```
MAE CV mặc định: 27.72
MAE test mặc định: 26.8 | MAE test tinh chỉnh: 25.85
```

Tinh chỉnh giảm MAE khoảng **1 nghìn USD** (CV: 27,72 → 26,49; test: 26,8 → 25,85). Khoảng 3,5%: có ích, nhưng không thần kỳ.

4.5. Học từ các lần thử

```
kq2 = pd.DataFrame(rs.cv_results_).sort_values("rank_test_score")
print(kq2[["param_learning_rate", "param_num_leaves",
```

```
    "mean_test_score"]].head(3).round(3))   # 3 tốt nhất
print(kq2[["param_learning_rate", "param_num_leaves",
"mean_test_score"]].tail(2).round(3))   # 2 tệ nhất
```

Kết quả mẫu:

```
    param_learning_rate  param_num_leaves  mean_test_score
24                0.033               110          -26.489
15                0.015                80          -27.085
20                0.063                43          -27.182
    param_learning_rate  param_num_leaves  mean_test_score
0                 0.254                68          -30.899
12                0.267               116          -30.945
```

Các bộ tốt nhất đều có learning_rate **nhỏ** (0,015–0,063). Các bộ tệ nhất có learning_rate **lớn** (~0,25). Khớp với bài 2.11. Thông tin này giúp **thu hẹp** phạm vi cho lần tìm kiếm tiếp theo (chiến lược "từ thô đến tinh").

---

5. Mẹo và bẫy

5.1. Mẹo

Mẹo	Giải thích
Từ thô đến tinh	Lần 1: phạm vi rộng, ít lần thử. Lần 2: thu hẹp quanh vùng tốt nhất
Thang log cho tham số "nhân"	C, alpha, learning_rate, gamma
Điểm tốt nhất ở rìa → mở rộng	Có thể còn giá trị tốt hơn ngoài phạm vi
Bắt đầu với Random Search, 20–60 lần thử	Thường đủ tốt
`n_jobs=-1`	Chạy song song
Boosting: dùng early stopping cho số cây	Không cần tìm `n_estimators` (bài 2.11)
Nhiều tham số, mô hình chậm	Thử **Optuna** (thư viện tìm kiếm thông minh, học từ các lần thử trước) hoặc `HalvingRandomSearchCV` (loại sớm các tổ hợp kém)

5.2. Bẫy

Bẫy 1 – Chỉnh trên tập test. Đã nói nhiều lần. Luôn dùng CV trên train.

Bẫy 2 – "Overfit" vào cross-validation. Thử **hàng nghìn** tổ hợp, bộ tốt nhất có thể tốt nhất **một phần nhờ may rủi** trên các fold cụ thể. Điểm `best_score_` vì vậy hơi **lạc quan**. Đó là lý do vẫn cần tập test cuối cùng.

(Muốn ước lượng hoàn toàn khách quan: **nested cross-validation**, một vòng CV bên ngoài bao quanh cả quá trình tìm kiếm.)

Bẫy 3 – Chênh lệch nhỏ hơn độ dao động. Bộ A: 0,781 ± 0,03; bộ B: 0,784 ± 0,03. Chênh 0,003 là **nhiễu**. Khi hai bộ ngang nhau, chọn bộ **đơn giản hơn** (ít lá hơn, regularization mạnh hơn): ít rủi ro overfit hơn.

Bẫy 4 – Tinh chỉnh quá sớm. Thứ tự ưu tiên: (1) dữ liệu sạch, (2) đặc trưng tốt, (3) chọn đúng loại mô hình, (4) **cuối cùng** mới tinh chỉnh.

---

6. Bài tập

1. Tinh chỉnh KNN cho dữ liệu Wine (bài 2.7) bằng `GridSearchCV`: `n_neighbors` từ 1 đến 30, `weights` là `"uniform"` hoặc `"distance"`. Nhớ pipeline có `StandardScaler`. Bao nhiêu lần huấn luyện? K tốt nhất là bao nhiêu?
2. Tinh chỉnh `Ridge` cho giá nhà (8 đặc trưng, có `PolynomialFeatures(3)` và `StandardScaler` trong pipeline, bài 2.6) với `alpha` theo `loguniform(1e-2, 1e3)`, 20 lần thử. So sánh với alpha = 10 mà bài 2.6 chọn bằng tay.
3. Tinh chỉnh Random Forest cho tín dụng (pipeline ở bài 2.17) với `max_features`, `min_samples_leaf`, `max_depth`, chấm điểm `roc_auc`. AUC CV cải thiện bao nhiêu so với mặc định (0,791)? Chênh lệch có lớn hơn độ dao động không?

Đáp án

> Mẹo Notion: chọn tiêu đề "Đáp án" → `Turn into → Toggle heading 3`.

1. 30 giá trị K × 2 cách trọng số = 60 tổ hợp × 5 fold = **300 lần huấn luyện** (KNN rất nhanh, vài giây).

```
from sklearn.datasets import load_wine
from sklearn.neighbors import KNeighborsClassifier
wine = load_wine(as_frame=True)
Xw_tr, Xw_te, yw_tr, yw_te = train_test_split(wine.data, wine.target,
test_size=0.3, random_state=42, stratify=wine.target)
pipe_knn = Pipeline([("chuan_hoa", StandardScaler()), ("knn",
KNeighborsClassifier())])
gs_knn = GridSearchCV(pipe_knn, {"knn__n_neighbors": list(range(1, 31)),
"knn__weights": ["uniform", "distance"]},
                      cv=StratifiedKFold(5, shuffle=True, random_state=42),
scoring="accuracy").fit(Xw_tr, yw_tr)
print(gs_knn.best_params_, round(gs_knn.best_score_, 3), round(gs_knn.score(Xw_te,
yw_te), 3))
```

Ghi lại K tốt nhất. Với chỉ 124 mẫu train, nhiều giá trị K cho điểm CV gần như bằng nhau: đó là bẫy 3, hãy chọn K **vừa phải** thay vì K cho điểm cao nhất tuyệt đối. 2. Gợi ý:

```
from sklearn.model_selection import RandomizedSearchCV
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import Ridge
pipe_r = Pipeline([("da_thuc", PolynomialFeatures(3)), ("chuan_hoa",
```

```
    StandardScaler()), ("ridge", Ridge())])
rs_r = RandomizedSearchCV(pipe_r, {"ridge__alpha": loguniform(1e-2, 1e3)},
n_iter=20, cv=cv_r,
                          scoring="r2", random_state=42, n_jobs=-1).fit(Xtr, ytr)
print(rs_r.best_params_, round(rs_r.best_score_, 3))
```

Bài 2.6 bài tập 3 cho thấy α = 1 tốt hơn α = 10 trên tập test; giờ bạn chọn α **đúng cách** (bằng CV, không nhìn test). Ghi lại α tốt nhất. 3. Gợi ý phạm vi: `{"mo_hinh__max_features": ["sqrt", 0.3, 0.5], "mo_hinh__min_samples_leaf": randint(1, 20), "mo_hinh__max_depth": [None, 5, 10, 20]}`, `n_iter=20`. Với 800 khách, cải thiện thường chỉ vài phần nghìn, **nhỏ hơn** độ dao động ±0,03 giữa các fold → không thể khẳng định là tốt hơn thật. Đây là kết luận trung thực và quan trọng: với dữ liệu nhỏ, đừng "vắt" quá nhiều từ tinh chỉnh.

---

7. Flashcard

Mặt trước	**Mặt sau**
Tham số vs siêu tham số?	Tham số: máy học (w, b). Siêu tham số: bạn chọn trước (alpha, K, C).
Grid Search là gì?	Thử mọi tổ hợp trong danh sách giá trị cho trước.
Nhược điểm Grid Search?	Số tổ hợp bùng nổ theo số tham số.
Vì sao Random Search thường tốt hơn?	Cùng số lần thử, khám phá nhiều giá trị khác nhau hơn của tham số quan trọng.
Tên siêu tham số trong pipeline?	`<tên bước>__<tham số>`, ví dụ `svm__C`.
Vì sao dùng `loguniform` cho learning_rate?	Mỗi bậc mười (0,001–0,01; 0,01–0,1) quan trọng như nhau.
`best_score_` có hoàn toàn khách quan?	Không, hơi lạc quan khi thử nhiều tổ hợp; vẫn cần tập test.
`refit=True` làm gì?	Tự huấn luyện lại bộ tốt nhất trên toàn bộ train.
Điểm tốt nhất nằm ở rìa lưới thì sao?	Mở rộng phạm vi về phía đó.
Thứ tự ưu tiên cải thiện mô hình?	Dữ liệu sạch → đặc trưng → loại mô hình → tinh chỉnh.

Bài tiếp theo: 2.19 – Project 1: Dự đoán giá nhà và chấm điểm tín dụng.